

The Complete Next.js Tutorial

From Zero to Advanced — A Step-by-Step Guide

Generated for your learning journey

2026

Table of Contents

1. Introduction to Next.js

- 1.1 What is Next.js?
- 1.2 Why use Next.js instead of plain React?
- 1.3 Rendering Strategies at a Glance
- 1.4 App Router vs Pages Router

2. Prerequisites & Environment Setup

3. Creating Your First Next.js Project

- 3.1 Scaffolding the app
- 3.2 Running the dev server
- 3.3 Project Structure Explained

4. File-based Routing (App Router)

- 4.1 Your first route
- 4.2 Nested Routes
- 4.3 Dynamic Routes
- 4.4 Catch-all Segments
- 4.5 Route Groups — organize without affecting the URL
- 4.6 Private Folders
- 4.7 Linking Between Pages
- 4.8 Programmatic Navigation

5. Layouts, Templates & Shared UI

- 5.1 The Root Layout
- 5.2 Nested Layouts
- 5.3 Templates
- 5.4 Loading UI (`loading.tsx`)
- 5.5 Error UI (`error.tsx`)
- 5.6 Not Found UI

6. Server Components vs Client Components

- 6.1 The default: Server Components

- 6.2 Client Components
- 6.3 Composing Server + Client Components
- 6.4 Quick Decision Table

7. Data Fetching

- 7.1 Fetching in Server Components
- 7.2 Caching Behavior — the part everyone gets confused by
- 7.3 Parallel vs Sequential Fetching
- 7.4 Streaming with Suspense
- 7.5 Static Site Generation with `generateStaticParams`
- 7.6 Client-side Data Fetching

8. Route Handlers (Building API Endpoints)

- 8.1 Basic GET handler
- 8.2 Handling multiple HTTP methods
- 8.3 Dynamic API Routes
- 8.4 Reading Query Parameters
- 8.5 Setting Headers & Status Codes

9. Server Actions & Forms

- 9.1 Defining a Server Action
- 9.2 Using it in a form
- 9.3 Revalidating Data After a Mutation
- 9.4 Handling Pending & Optimistic UI
- 9.5 Calling Server Actions from Event Handlers

10. Styling in Next.js

- 10.1 Global CSS
- 10.2 CSS Modules — scoped by default
- 10.3 Tailwind CSS
- 10.4 CSS-in-JS (e.g. styled-components)

11. Images, Fonts & Metadata (SEO)

- 11.1 The `<Image>` Component
- 11.2 Font Optimization
- 11.3 Metadata for SEO

12. Environment Variables

13. Middleware

14. Authentication (Overview)

15. State Management

- 15.1 Local state — `useState` / `useReducer`
- 15.2 Sharing state across components — Context
- 15.3 Global client state — Zustand (lightweight alternative to Redux)
- 15.4 Server state — usually shouldn't be "state" at all

16. Advanced Routing Patterns

- 16.1 Parallel Routes
- 16.2 Intercepting Routes
- 16.3 Route Handlers + Streaming Responses

17. Caching Deep Dive

- 17.1 Revalidating Data
- 17.2 Opting out of caching for a whole route

18. Performance Optimization Checklist

19. Testing

- 19.1 Unit/Component Tests — Vitest + React Testing Library
- 19.2 End-to-End Tests — Playwright

20. Deployment

- 20.1 Deploying to Vercel (simplest path)
- 20.2 Self-Hosting with Docker
- 20.3 Other Hosting Options

21. Recommended Project Structure (Real-World)

22. Where to Go Next

- [Quick Reference Cheat Sheet](#)

1. Introduction to Next.js

1.1 What is Next.js?

Next.js is a **React framework** created by Vercel that adds the production-grade features React alone doesn't provide out of the box: routing, server rendering, bundling, data fetching conventions, and performance optimizations.

Think of it this way:

- **React** is a library for building UI components.
- **Next.js** is a full framework built *on top of* React that decides how your app is structured, rendered, and shipped.

1.2 Why use Next.js instead of plain React?

Feature	Plain React (Vite/CRA)	Next.js
Routing	Manual (react-router)	Built-in, file-based
Server Rendering (SSR)	Manual setup	Built-in
Static Site Generation	Manual setup	Built-in
API endpoints	Separate backend needed	Built-in Route Handlers
Image optimization	Manual	Built-in <code><Image></code>
SEO	Harder (CSR by default)	Easy (SSR/SSG by default)
Code splitting	Manual config	Automatic

1.3 Rendering Strategies at a Glance

Next.js supports multiple rendering strategies, and you can mix them in the same app:

- **SSG (Static Site Generation)** — HTML built at *build time*. Fastest, best for blogs/marketing pages.
- **SSR (Server-Side Rendering)** — HTML built on *every request*. Best for personalized/frequently changing data.

- **ISR (Incremental Static Regeneration)** — Static pages that regenerate on a timer. Best of both worlds.
- **CSR (Client-Side Rendering)** — Rendered in the browser, like traditional React. Best for highly interactive, private dashboards.

We'll explore all four hands-on later in this guide.

1.4 App Router vs Pages Router

Next.js has two routing systems:

- **Pages Router** (`/pages` directory) — the original system, still supported.
- **App Router** (`/app` directory) — the modern system (Next.js 13+), built on **React Server Components**.

This tutorial uses the App Router, since it's the recommended approach for all new projects going forward.

2. Prerequisites & Environment Setup

Before starting, make sure you have:

1. **Node.js 18.18 or later** (Node 20 LTS recommended). Check with:

```
node -v  
npm -v
```

1. A code editor — **VS Code** is recommended, with these extensions:
 - ES7+ React/Redux snippets
 - Tailwind CSS IntelliSense (if using Tailwind)
 - Prettier
2. Basic knowledge of **JavaScript/TypeScript** and **React** (components, props, hooks like `useState` / `useEffect`).

3. Creating Your First Next.js Project

3.1 Scaffolding the app

Open a terminal and run:

```
npx create-next-app@latest my-nextjs-app
```

You'll be asked a series of setup questions:

```
✓ Would you like to use TypeScript?           → Yes
✓ Would you like to use ESLint?                 → Yes
✓ Would you like to use Tailwind CSS?           → Yes
✓ Would you like your code inside a `src/` directory? → Yes
✓ Would you like to use App Router?             → Yes
✓ Would you like to use Turbopack for next dev? → Yes
✓ Would you like to customize the import alias? → No
```

3.2 Running the dev server

```
cd my-nextjs-app
npm run dev
```

Visit **<http://localhost:3000>** — you should see the default Next.js welcome page. That's your app, live, with hot reloading enabled.

3.3 Project Structure Explained

```
my-nextjs-app/
├── src/
│   └── app/
│       ├── favicon.ico
│       ├── globals.css
│       ├── layout.tsx      # Root layout – wraps every page
│       └── page.tsx        # Home page → route "/"
├── public/                 # Static assets (images, fonts, etc.)
├── next.config.ts          # Next.js configuration
├── tsconfig.json           # TypeScript config
├── package.json
└── tailwind.config.ts
```

Key idea: **inside `app/`, folders define URL routes, and special files define UI for that route.**

File name	Purpose
<code>page.tsx</code>	The unique UI for a route (makes a folder publicly accessible)
<code>layout.tsx</code>	Shared UI that wraps child pages/layouts
<code>loading.tsx</code>	Loading UI (Suspense boundary)
<code>error.tsx</code>	Error UI (Error boundary)
<code>not-found.tsx</code>	UI for 404s
<code>route.ts</code>	API endpoint for that path
<code>template.tsx</code>	Like layout, but re-mounts on navigation

4. File-based Routing (App Router)

4.1 Your first route

Every `page.tsx` inside `app/` becomes a route based on its folder path.

```
app/
├── page.tsx      → /
├── about/
│   └── page.tsx  → /about
└── blog/
    └── page.tsx   → /blog
```

Create the about page:

```
// app/about/page.tsx
export default function AboutPage() {
  return (
    <main>
      <h1>About Us</h1>
      <p>This page is rendered from app/about/page.tsx</p>
    </main>
  );
}
```

Visit `http://localhost:3000/about` — it just works. No router config needed.

4.2 Nested Routes

```
app/
└── dashboard/
    ├── page.tsx      → /dashboard
    └── settings/
        └── page.tsx   → /dashboard/settings
```

4.3 Dynamic Routes

Wrap a folder name in square brackets to create a dynamic segment:

```
app/  
└─ blog/  
  └─ [slug]/  
    └─ page.tsx      → /blog/:slug
```

```
// app/blog/[slug]/page.tsx  
export default async function BlogPostPage({  
  params,  
}: {  
  params: Promise<{ slug: string }>;  
}) {  
  const { slug } = await params;  
  return <h1>Blog Post: {slug}</h1>;  
}
```

Visiting `/blog/hello-world` renders “Blog Post: hello-world”.

Note: From Next.js 15 onward, `params` and `searchParams` are **Promises** and must be awaited.

4.4 Catch-all Segments

```
app/shop/[...slug]/page.tsx    → matches /shop/a, /shop/a/b, /shop/a/b/c ...  
app/shop/[ [...slug] ]/page.tsx → also matches /shop (optional catch-all)
```

```
export default async function ShopPage({  
  params,  
}: {  
  params: Promise<{ slug?: string[] }>;  
}) {  
  const { slug } = await params;  
  return <p>Path segments: {slug?.join(" / ") ?? "none"}</p>;  
}
```

4.5 Route Groups — organize without affecting the URL

Wrap a folder name in parentheses to group routes *without* adding a URL segment:

```
app/
├─ (marketing)/
│   ├─ page.tsx          → /
│   └─ pricing/page.tsx → /pricing
└─ (app)/
    └─ dashboard/page.tsx → /dashboard
```

This is purely organizational — great for separating a marketing site's layout from an app's layout while sharing the same domain.

4.6 Private Folders

Prefix a folder with an underscore (e.g. `_components`) to opt it **out** of routing entirely — Next.js will never treat it as a route, useful for co-locating helper files.

4.7 Linking Between Pages

Always use the `<Link>` component instead of `<a>` for internal navigation — it enables client-side transitions and prefetching.

```
import Link from "next/link";

export default function Nav() {
  return (
    <nav>
      <Link href="/">Home</Link>
      <Link href="/about">About</Link>
      <Link href="/blog/hello-world">First Post</Link>
    </nav>
  );
}
```

4.8 Programmatic Navigation

```
"use client";
import { useRouter } from "next/navigation";

export default function LoginButton() {
  const router = useRouter();

  function handleLogin() {
    // ...auth logic
    router.push("/dashboard");
  }

  return <button onClick={handleLogin}>Log in</button>;
}
```

Other useful hooks from `next/navigation` :

- `usePathname()` — current URL path
- `useSearchParams()` — read query string params
- `useParams()` — read dynamic route params in a Client Component

5. Layouts, Templates & Shared UI

5.1 The Root Layout

Every App Router project **must** have a root layout at `app/layout.tsx`. It wraps every single page in your app.

```
// app/layout.tsx
import "../globals.css";

export const metadata = {
  title: "My Next.js App",
  description: "Built while learning Next.js",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body>
        <header>My Site Header</header>
        {children}
        <footer>© 2026 My Site</footer>
      </body>
    </html>
  );
}
```

Notice it renders `<html>` and `<body>` tags directly — this is the *only* place that should happen.

5.2 Nested Layouts

Any folder can have its own `layout.tsx`, which wraps only the pages inside that folder (and nests inside parent layouts).

```

app/
├─ layout.tsx           # wraps everything
├─ dashboard/
│   ├─ layout.tsx       # wraps only /dashboard/*
│   ├─ page.tsx
│   └─ settings/page.tsx

```

```

// app/dashboard/layout.tsx
export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div style={{ display: "flex" }}>
      <aside>Sidebar: Dashboard Nav</aside>
      <section>{children}</section>
    </div>
  );
}

```

Layouts **persist across navigation** within their scope — they don't re-render when you move between child pages, which preserves state (like an open sidebar or a video player).

5.3 Templates

`template.tsx` looks identical to a layout but creates a **new instance on every navigation** (state is reset, effects re-run). Use it when you *want* a fresh mount — e.g., enter/exit animations.

5.4 Loading UI (`loading.tsx`)

Add a `loading.tsx` next to any `page.tsx` to show instant loading states while the page's data loads — Next.js wraps the page in a React `<Suspense>` boundary automatically.

```

// app/dashboard/loading.tsx
export default function Loading() {
  return <p>Loading dashboard...</p>;
}

```

5.5 Error UI (`error.tsx`)

```
// app/dashboard/error.tsx
"use client"; // error boundaries must be Client Components

import { useEffect } from "react";

export default function Error({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  useEffect(() => {
    console.error(error);
  }, [error]);

  return (
    <div>
      <h2>Something went wrong!</h2>
      <button onClick={() => reset()}>Try again</button>
    </div>
  );
}
```

5.6 Not Found UI

```
// app/blog/[slug]/not-found.tsx
export default function NotFound() {
  return <h2>Post not found.</h2>;
}
```

Trigger it manually from a page:

```
import { notFound } from "next/navigation";

async function getPost(slug: string) {
  const post = await db.post.find(slug);
  if (!post) notFound();
  return post;
}
```

6. Server Components vs Client Components

This is the single most important mental model shift in the App Router.

6.1 The default: Server Components

Every component in `app/` is a **Server Component by default**. It runs only on the server, never ships JS to the browser, and can directly access backend resources (databases, file systems, secrets).

```
// This is a Server Component – no directive needed
async function ProductList() {
  const res = await fetch("https://api.example.com/products");
  const products = await res.json();

  return (
    <ul>
      {products.map((p: { id: string; name: string }) => (
        <li key={p.id}>{p.name}</li>
      ))}
    </ul>
  );
}

export default ProductList;
```

Benefits: - Zero client-side JS bundle cost. - Can `await` data directly in the component body.
- Keeps secrets (API keys, DB credentials) off the client.

6.2 Client Components

Add `"use client"` at the very top of a file to opt into the traditional React model: state, effects, browser APIs, event handlers.

```
// app/components/Counter.tsx
"use client";

import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Use a Client Component when you need:

- `useState` , `useReducer` , `useEffect`
- Event listeners (`onClick` , `onChange` , ...)
- Browser-only APIs (`localStorage` , `window`)
- Third-party libraries that depend on the above (some charting libs, etc.)

6.3 Composing Server + Client Components

The golden rule: **Server Components can render Client Components, but Client Components cannot import Server Components directly.** Instead, pass Server Components down as `children` or props.

```
// app/page.tsx (Server Component)
import Counter from "../components/Counter"; // Client Component
import ProductList from "../components/ProductList"; // Server Component

export default function HomePage() {
  return (
    <main>
      <ProductList />    {/* rendered on the server */}
      <Counter />        {/* hydrated in the browser */}
    </main>
  );
}
```

```
// Passing a Server Component as a child into a Client Component
"use client";
export default function ClientWrapper({ children }: { children:
React.ReactNode }) {
  return <div className="border p-4">{children}</div>;
}
```

```
// app/page.tsx
import ClientWrapper from "../ClientWrapper";
import ServerWidget from "../ServerWidget";

export default function Page() {
  return (
    <ClientWrapper>
      <ServerWidget /> { /* still renders on the server! */ }
    </ClientWrapper>
  );
}
```

6.4 Quick Decision Table

Need	Component type
Fetch data from a database/API	Server
Use hooks (<code>useState</code> , <code>useEffect</code>)	Client
Handle click/input events	Client
Keep an API key secret	Server
Use <code>window</code> , <code>localStorage</code>	Client
Render static/markup-heavy content	Server

7. Data Fetching

7.1 Fetching in Server Components

Because Server Components can be `async`, data fetching is refreshingly simple — no `useEffect`, no loading state boilerplate:

```
async function getUser(id: string) {
  const res = await fetch(`https://api.example.com/users/${id}`);
  if (!res.ok) throw new Error("Failed to fetch user");
  return res.json();
}

export default async function UserPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const user = await getUser(id);

  return <h1>Welcome, {user.name}</h1>;
}
```

7.2 Caching Behavior — the part everyone gets confused by

Next.js extends the native `fetch()` with a `cache` option:

```
// Cached indefinitely (like getStaticProps) – this is now opt-in in Next.js
15 fetch(url, { cache: "force-cache" });

// Never cached, always fresh (like getServerSideProps)
fetch(url, { cache: "no-store" });

// Revalidate every 60 seconds (Incremental Static Regeneration)
fetch(url, { next: { revalidate: 60 } });
```

Next.js 15+ note: `fetch` requests are **not cached by default** anymore (this changed from earlier versions). Add `cache: "force-cache"` explicitly when you want static caching behavior.

7.3 Parallel vs Sequential Fetching

Sequential (slower — waterfall):

```
export default async function Page() {
  const user = await getUser();          // waits...
  const posts = await getPosts(user.id); // ...then waits again
  return <Profile user={user} posts={posts} />;
}
```

Parallel (faster):

```
export default async function Page() {
  const userPromise = getUser();
  const postsPromise = getPosts();

  const [user, posts] = await Promise.all([userPromise, postsPromise]);
  return <Profile user={user} posts={posts} />;
}
```

7.4 Streaming with Suspense

Wrap slow components in `<Suspense>` so the rest of the page can render immediately while that piece streams in later:

```
import { Suspense } from "react";

export default function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>
      <Suspense fallback=<p>Loading stats...</p>>
        <SlowStatsWidget />
      </Suspense>
    </div>
  );
}

async function SlowStatsWidget() {
  const stats = await getStatsAfterDelay(); // takes 3 seconds
  return <StatsChart data={stats} />;
}
```

7.5 Static Site Generation with `generateStaticParams`

For dynamic routes you want pre-rendered at build time:

```
// app/blog/[slug]/page.tsx
export async function generateStaticParams() {
  const posts = await getAllPosts();
  return posts.map((post) => ({ slug: post.slug }));
}

export default async function BlogPost({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params;
  const post = await getPost(slug);
  return <article>{post.content}</article>;
}
```

This is Next.js's version of **SSG** — every slug returned gets its own static HTML file at build time.

7.6 Client-side Data Fetching

Sometimes you *do* want to fetch on the client — e.g., data that changes based on user interaction after the initial load. Use a library like **SWR** or **TanStack Query** rather than raw

`useEffect` + `fetch`:

```
"use client";
import useSWR from "swr";

const fetcher = (url: string) => fetch(url).then((res) => res.json());

export default function LiveScore() {
  const { data, error, isLoading } = useSWR("/api/score", fetcher, {
    refreshInterval: 5000, // poll every 5s
  });

  if (isLoading) return <p>Loading...</p>;
  if (error) return <p>Failed to load.</p>;
  return <p>Score: {data.score}</p>;
}
```

8. Route Handlers (Building API Endpoints)

Route Handlers let you build API endpoints directly inside `app/`, replacing the old `pages/api` approach.

8.1 Basic GET handler

```
// app/api/hello/route.ts
import { NextResponse } from "next/server";

export async function GET() {
  return NextResponse.json({ message: "Hello, world!" });
}
```

Visiting `/api/hello` returns the JSON response.

8.2 Handling multiple HTTP methods

```
// app/api/todos/route.ts
import { NextResponse } from "next/server";

let todos = [{ id: 1, text: "Learn Next.js" }];

export async function GET() {
  return NextResponse.json(todos);
}

export async function POST(request: Request) {
  const body = await request.json();
  const newTodo = { id: Date.now(), text: body.text };
  todos.push(newTodo);
  return NextResponse.json(newTodo, { status: 201 });
}
```

8.3 Dynamic API Routes

```
// app/api/todos/[id]/route.ts
import { NextResponse } from "next/server";

export async function DELETE(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  // ...delete logic
  return NextResponse.json({ deleted: id });
}
```

8.4 Reading Query Parameters

```
// app/api/search/route.ts
import { NextResponse } from "next/server";

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const query = searchParams.get("q");
  return NextResponse.json({ results: `Searching for: ${query}` });
}
```

8.5 Setting Headers & Status Codes

```
export async function GET() {
  return new NextResponse(JSON.stringify({ ok: true }), {
    status: 200,
    headers: { "Content-Type": "application/json", "X-Custom-Header":
"abc" },
  });
}
```

9. Server Actions & Forms

Server Actions let you call server-side functions directly from Client (or Server) Components — no manually built API route required. This is one of the App Router's most powerful features.

9.1 Defining a Server Action

```
// app/actions.ts
"use server";

export async function createTodo(formData: FormData) {
  const text = formData.get("text") as string;
  await db.todo.create({ text });
}
```

9.2 Using it in a form

```
// app/todos/page.tsx
import { createTodo } from "../actions";

export default function TodosPage() {
  return (
    <form action={createTodo}>
      <input type="text" name="text" placeholder="New todo" />
      <button type="submit">Add</button>
    </form>
  );
}
```

Notice: **no** `onSubmit`, **no** `fetch`, **no API route**. The form works even with JavaScript disabled (progressive enhancement), and Next.js wires the action up automatically.

9.3 Revalidating Data After a Mutation

```
"use server";
import { revalidatePath } from "next/cache";

export async function createTodo(formData: FormData) {
  const text = formData.get("text") as string;
  await db.todo.create({ text });
  revalidatePath("/todos"); // tell Next.js this page's cache is stale
}
```

9.4 Handling Pending & Optimistic UI

```
"use client";
import { useFormStatus } from "react-dom";

function SubmitButton() {
  const { pending } = useFormStatus();
  return (
    <button type="submit" disabled={pending}>
      {pending ? "Adding..." : "Add Todo"}
    </button>
  );
}
```

```

"use client";
import { useOptimistic } from "react";

export default function TodoList({ todos }: { todos: { id: number; text: string }[] }) {
  const [optimisticTodos, addOptimisticTodo] = useOptimistic(
    todos,
    (state, newTodo: string) => [...state, { id: Date.now(), text: newTodo }]
  );

  async function handleSubmit(formData: FormData) {
    const text = formData.get("text") as string;
    addOptimisticTodo(text);
    await createTodo(formData);
  }

  return (
    <>
      <form action={handleSubmit}>
        <input name="text" />
        <button type="submit">Add</button>
      </form>
      <ul>
        {optimisticTodos.map((t) => (
          <li key={t.id}>{t.text}</li>
        ))}
      </ul>
    </>
  );
}

```

9.5 Calling Server Actions from Event Handlers

Server Actions aren't limited to `<form>` — call them like normal async functions:

```
"use client";
import { deleteTodo } from "../actions";

export default function DeleteButton({ id }: { id: number }) {
  return (
    <button onClick={() => deleteTodo(id)}>
      Delete
    </button>
  );
}
```

10. Styling in Next.js

10.1 Global CSS

```
/* app/globals.css */
:root {
  --color-primary: #0070f3;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
}
```

Import it once, in the root layout only:

```
// app/layout.tsx
import "../globals.css";
```

10.2 CSS Modules — scoped by default

```
/* app/components/Button.module.css */
.button {
  background: var(--color-primary);
  color: white;
  padding: 0.5rem 1rem;
  border-radius: 6px;
}
```

```
import styles from "../Button.module.css";

export default function Button({ children }: { children: React.ReactNode }) {
  return <button className={styles.button}>{children}</button>;
}
```

10.3 Tailwind CSS

`create-next-app` can wire this up for you automatically. Usage is just utility classes:

```
export default function Card() {
  return (
    <div className="rounded-lg shadow-md p-6 bg-white hover:shadow-lg
transition-shadow">
      <h2 className="text-xl font-bold text-gray-800">Card Title</h2>
      <p className="text-gray-600 mt-2">Card description text.</p>
    </div>
  );
}
```

10.4 CSS-in-JS (e.g. styled-components)

Supported, but requires a small config for Server Components compatibility since these libraries generate styles at runtime — check the library's Next.js App Router guide before adopting. For new projects, **Tailwind or CSS Modules are the recommended default** because they work seamlessly with Server Components.

11. Images, Fonts & Metadata (SEO)

11.1 The `<Image>` Component

```
import Image from "next/image";

export default function Avatar() {
  return (
    <Image
      src="/profile.png"
      alt="User profile picture"
      width={128}
      height={128}
      priority // preload for above-the-fold images
    />
  );
}
```

Benefits over `<img>`: automatic lazy loading, responsive `srcset` generation, and modern format conversion (WebP/AVIF) on the fly.

For remote images, allow the domain in `next.config.ts`:

```
// next.config.ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [{ hostname: "images.example.com" }],
  },
};

export default nextConfig;
```

11.2 Font Optimization

```
// app/layout.tsx
import { Inter } from "next/font/google";

const inter = Inter({ subsets: ["latin"] });

export default function RootLayout({ children }: { children:
React.ReactNode }) {
  return (
    <html lang="en" className={inter.className}>
      <body>{children}</body>
    </html>
  );
}
```

`next/font` self-hosts Google Fonts at build time — no runtime request to Google, no layout shift.

11.3 Metadata for SEO

Static metadata:

```
// app/about/page.tsx
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "About Us | My Site",
  description: "Learn more about our company.",
  openGraph: {
    title: "About Us",
    images: ["/og-image.png"],
  },
};

export default function AboutPage() {
  return <h1>About</h1>;
}
```

Dynamic metadata (e.g. based on fetched data):

```
// app/blog/[slug]/page.tsx
import type { Metadata } from "next";

export async function generateMetadata({
  params,
}: {
  params: Promise<{ slug: string }>;
}): Promise<Metadata> {
  const { slug } = await params;
  const post = await getPost(slug);

  return {
    title: post.title,
    description: post.excerpt,
  };
}
```

Metadata defined in a layout merges with (and is overridden by) metadata from nested pages — the deepest, most specific definition wins for conflicting fields.

12. Environment Variables

```
.env.local      # local secrets, never committed
.env.production # production defaults
.env.development # development defaults
```

```
# .env.local
DATABASE_URL=postgres://user:pass@localhost:5432/db
NEXT_PUBLIC_ANALYTICS_ID=abc123
```

- Variables **without** the `NEXT_PUBLIC_` prefix are only available on the **server** (safe for secrets).
- Variables **with** the `NEXT_PUBLIC_` prefix are inlined into the **client bundle** at build time — never put secrets here.

```
// Server Component – safe to read secret vars
const dbUrl = process.env.DATABASE_URL;

// Client Component – only NEXT_PUBLIC_ vars are visible
"use client";
const analyticsId = process.env.NEXT_PUBLIC_ANALYTICS_ID;
```

Always add `.env.local` to `.gitignore` (create-next-app does this for you already).

13. Middleware

Middleware runs **before** a request completes, at the edge — perfect for auth checks, redirects, A/B testing, or geolocation logic.

```
// middleware.ts (at the project root, next to app/)
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  const token = request.cookies.get("session")?.value;

  if (!token && request.nextUrl.pathname.startsWith("/dashboard")) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/dashboard/:path*"],
};
```

Common middleware use cases:

- Redirect unauthenticated users away from protected routes.
- Rewrite requests for A/B testing (`NextResponse.rewrite`).
- Add security headers to every response.
- Internationalization — detect locale from cookies/headers and redirect.

14. Authentication (Overview)

Next.js has no built-in auth system by design — you bring your own. The most common approaches:

Approach	When to use
Auth.js (NextAuth)	OAuth providers (Google, GitHub) + credentials, session cookies
Clerk / Auth0	Managed, hosted auth with prebuilt UI
Custom JWT + Middleware	Full control, simple session needs

A minimal Auth.js setup looks like:

```
// app/api/auth/[...nextauth]/route.ts
import NextAuth from "next-auth";
import GitHub from "next-auth/providers/github";

const handler = NextAuth({
  providers: [
    GitHub({
      clientId: process.env.GITHUB_ID!,
      clientSecret: process.env.GITHUB_SECRET!,
    }),
  ],
});

export { handler as GET, handler as POST };
```

```
// Checking a session in a Server Component
import { auth } from "@/auth";

export default async function Dashboard() {
  const session = await auth();
  if (!session) return <p>Please sign in.</p>;
  return <p>Welcome, {session.user?.name}</p>;
}
```

Pair this with the Middleware pattern from Section 13 to protect entire route groups.

15. State Management

15.1 Local state — `useState` / `useReducer`

Fine for component-local UI state (toggles, form inputs, tabs).

15.2 Sharing state across components — Context

```
"use client";
import { createContext, useContext, useState } from "react";

const CartContext = createContext<{
  items: string[];
  addItem: (item: string) => void;
} | null>(null);

export function CartProvider({ children }: { children: React.ReactNode }) {
  const [items, setItems] = useState<string[]>([]);
  const addItem = (item: string) => setItems((prev) => [...prev, item]);

  return (
    <CartContext.Provider value={{ items, addItem }}>
      {children}
    </CartContext.Provider>
  );
}

export function useCart() {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error("useCart must be used within CartProvider");
  return ctx;
}
```

Context providers are Client Components. Wrap only the subtree that needs the state — not your entire root layout — to avoid opting more of the tree into client rendering than necessary.

15.3 Global client state — Zustand (lightweight alternative to Redux)

```
// store/useCartStore.ts
"use client";
import { create } from "zustand";

interface CartState {
  items: string[];
  addItem: (item: string) => void;
}

export const useCartStore = create<CartState>((set) => ({
  items: [],
  addItem: (item) => set((state) => ({ items: [...state.items, item] })),
}));
```

```
"use client";
import { useCartStore } from "@/store/useCartStore";

export default function CartBadge() {
  const items = useCartStore((state) => state.items);
  return <span>{items.length} items</span>;
}
```

15.4 Server state — usually shouldn't be “state” at all

A common mistake: fetching server data into `useState` + `useEffect`. In the App Router, prefer fetching directly in a Server Component and passing data down as props — there's often no client “state” needed at all.

16. Advanced Routing Patterns

16.1 Parallel Routes

Render multiple independent pages in the same layout simultaneously, using named “slots” (`@folder`):

```
app/  
├─ dashboard/  
│  ├─ layout.tsx  
│  ├─ @analytics/  
│  │  └─ page.tsx  
│  └─ @team/  
│     └─ page.tsx
```

```
// app/dashboard/layout.tsx  
export default function DashboardLayout({  
  children,  
  analytics,  
  team,  
}: {  
  children: React.ReactNode;  
  analytics: React.ReactNode;  
  team: React.ReactNode;  
}) {  
  return (  
    <div className="grid grid-cols-2">  
      {children}  
      {analytics}  
      {team}  
    </div>  
  );  
}
```

Each slot has its own `loading.tsx` and `error.tsx`, so slow widgets don't block the rest of the dashboard.

16.2 Intercepting Routes

Show a route in a modal over the current page, while still supporting a full page reload/direct link to the same content (e.g., Instagram-style photo modals).

```
app/
├─ feed/
│   └─ page.tsx
├─ photo/
│   └─ [id]/
│       └─ page.tsx      → full page view of a photo
└─ feed/
    └─ (..)photo/
        └─ [id]/
            └─ page.tsx   → intercepted modal view, shown over /feed
```

The `(..)` convention matches one level up in the URL — `(.)`, `(..)`, `(...)` intercept sibling, parent, and root-level segments respectively.

16.3 Route Handlers + Streaming Responses

```
// app/api/stream/route.ts
export async function GET() {
  const stream = new ReadableStream({
    async start(controller) {
      for (let i = 0; i < 5; i++) {
        controller.enqueue(`chunk ${i}\n`);
        await new Promise((r) => setTimeout(r, 1000));
      }
      controller.close();
    },
  });

  return new Response(stream, {
    headers: { "Content-Type": "text/plain" },
  });
}
```

17. Caching Deep Dive

Next.js has **four distinct caching layers** — understanding them is key to debugging “why isn’t my data updating?”:

Cache	What it stores	Where	Duration
Request Memoization	Duplicate <code>fetch</code> calls in one render pass	Server, per-request	Single request
Data Cache	Results of <code>fetch</code> (with caching enabled)	Server, persistent	Until revalidated/ deployed
Full Route Cache	Rendered HTML + RSC payload of static routes	Server, persistent	Until revalidated/ deployed
Router Cache	Visited route segments	Client (browser), in-memory	Session / time-based

17.1 Revalidating Data

```
// Time-based revalidation
fetch(url, { next: { revalidate: 3600 } }); // every hour

// Tag-based revalidation – invalidate by tag, from anywhere
fetch(url, { next: { tags: ["products"] } });
```

```
"use server";
import { revalidateTag, revalidatePath } from "next/cache";

export async function updateProduct() {
  await db.product.update(/* ... */);
  revalidateTag("products"); // invalidates every fetch tagged "products"
  revalidatePath("/products"); // invalidates a specific route's cache
}
```

17.2 Opting out of caching for a whole route

```
// app/dashboard/page.tsx
export const dynamic = "force-dynamic"; // always render fresh, per request
```

Other route segment config options:

```
export const revalidate = 60;           // ISR: revalidate every 60s
export const fetchCache = "force-no-store"; // disable data cache for this
route
```

18. Performance Optimization Checklist

1. **Use Server Components by default** — only add `"use client"` where interactivity is truly needed.
2. **Colocate `"use client"` at the leaves** of your component tree, not the root, to minimize the JS bundle shipped to the browser.
3. **Use `<Suspense>` boundaries** to stream slow data instead of blocking the whole page.
4. **Use `next/image`** for every image — never a raw `<img>` for content images.
5. **Use `next/font`** instead of a `<link>` tag to Google Fonts.
6. **Use `next/dynamic`** to lazy-load heavy client components:

```
import dynamic from "next/dynamic";

const HeavyChart = dynamic(() => import("./HeavyChart"), {
  loading: () => <p>Loading chart...</p>,
  ssr: false, // skip server rendering for browser-only libraries
});
```

1. **Run `next build`** locally and check the output bundle sizes/route types before deploying:

```
npm run build
```

Route (app)	Size	First Load JS
└ ○ /	142 B	87.3 kB
└ • /blog/[slug]	1.2 kB	89.1 kB
└ f /dashboard	3.4 kB	94.7 kB

○ (Static)	prerendered as static content
• (SSG)	prerendered with <code>generateStaticParams</code>
f (Dynamic)	server-rendered on demand

19. Testing

19.1 Unit/Component Tests — Vitest + React Testing Library

```
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
```

```
// Button.test.tsx
import { render, screen } from "@testing-library/react";
import { describe, it, expect } from "vitest";
import Button from "./Button";

describe("Button", () => {
  it("renders children text", () => {
    render(<Button>Click me</Button>);
    expect(screen.getByText("Click me")).toBeInTheDocument();
  });
});
```

19.2 End-to-End Tests — Playwright

```
npm init playwright@latest
```

```
// e2e/home.spec.ts
import { test, expect } from "@playwright/test";

test("homepage has a title", async ({ page }) => {
  await page.goto("http://localhost:3000");
  await expect(page).toHaveTitle(/My Next.js App/);
});
```

20. Deployment

20.1 Deploying to Vercel (simplest path)

```
npm install -g vercel
vercel
```

Follow the prompts — Vercel (the creator of Next.js) auto-detects the framework and configures everything, including automatic preview deployments per Git branch/PR.

20.2 Self-Hosting with Docker

```
# Dockerfile
FROM node:20-alpine AS base

FROM base AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM base AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM base AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static

EXPOSE 3000
CMD ["node", "server.js"]
```

Enable the standalone output mode first:

```
// next.config.ts
const nextConfig = {
  output: "standalone",
};
export default nextConfig;
```

```
docker build -t my-nextjs-app .
docker run -p 3000:3000 my-nextjs-app
```

20.3 Other Hosting Options

- **Netlify, AWS Amplify, Railway, Render** — all support Next.js via adapters or Docker.
- **Self-managed Node server** — run `npm run build && npm start` behind Nginx/PM2.

21. Recommended Project Structure (Real-World)

```
src/
├─ app/
│   ├─ (marketing)/      # public pages, own layout
│   ├─ (app)/            # authenticated app, own layout
│   │   └─ dashboard/
│   ├─ api/              # route handlers
│   ├─ layout.tsx
│   └─ globals.css
├─ components/          # shared, reusable UI components
│   ├─ ui/               # generic building blocks (Button, Input...)
│   └─ features/         # feature-specific components
├─ lib/                 # utilities, db client, helpers
├─ hooks/               # custom React hooks
├─ actions/             # server actions grouped by feature
├─ types/               # shared TypeScript types
└─ styles/
```

22. Where to Go Next

- **Official docs:** nextjs.org/docs — the single best source of truth, updated continuously.
- Build a real project: a blog with MDX, an e-commerce storefront, or a SaaS dashboard.
- Explore **Turbopack** (now stable for `next dev` and increasingly for builds) for faster local development.
- Learn **React Server Components** more deeply — they're the foundation this entire model is built on.
- Study **Partial Prerendering (PPR)** — Next.js's experimental feature to combine a static shell with dynamic, streamed islands on the same page.

Quick Reference Cheat Sheet

I want to...	Use
Create a new page	<code>app/route/page.tsx</code>
Share UI across pages	<code>layout.tsx</code>
Fetch data on the server	<code>async</code> Server Component + <code>fetch</code>
Build an API endpoint	<code>route.ts</code> with <code>GET</code> / <code>POST</code> /etc.
Mutate data from a form	Server Action (<code>"use server"</code>)
Add interactivity	<code>"use client"</code> + hooks
Protect a route	<code>middleware.ts</code>
Optimize an image	<code>next/image</code>
Pre-render dynamic paths	<code>generateStaticParams</code>
Show a loading spinner	<code>loading.tsx</code> or <code><Suspense></code>
Handle a 404	<code>not-found.tsx</code>
Set page title/SEO tags	<code>metadata</code> export or <code>generateMetadata</code>

Congratulations — you've gone from a blank terminal to understanding Server Components, Server Actions, caching layers, and deployment. The best next step now is to **build something real** with what you've learned.